Comparative Analysis of Methods for Detecting and Classifying Insects from Agricultural Environments

Chong Pang
Z5524074
University of New South Wales

Mengze Ge
Z5635577
University of New South Wales

Junzhe Zha
Z5513608
University of New South Wales

Songyu Qi
Z5536858
University of New South Wales

Yunmiao Qi
Z5571406
University of New South Wales

***Abstract*—Agriculture plays an important role in human society, but pests cause immense damage. To further protect agricultural development, this research try a series of computer vision methods for insect detection and recognition. Our work is based on the AgroPest-12 public dataset, which contains 12 insect categories. We first used machine learning for preliminary experiments, extracting features based on SIFT features and the Bag of Visual Words (BoVW) model. After feature extraction, we tested five classifiers (SVM, KNN, Random Forest, XGBoost, and MLP). Then we focus on end-to-end deep learning neural network architectures. Implemented a two-stage detector, Faster R-CNN, based on ResNet-50-FPN, finding that while training was very slow, but its accuracy was significantly higher than MLP. Therefore, to further improve training efficiency, we replaced ResNet-50-FPN with a single-stage detector, RetinaNet, finding that it not only trained faster but also achieved significantly higher accuracy. Finally, to further optimize performance, we tested the most famous single-stage detector YOLO. Comprehensive experimental results show that the YOLO architecture achieves the best overall balance between mean average accuracy (mAP) and real-time inference speed.**

***Index Terms*—Machine Learning, MLP, Faster R-CNN, RetinaNet, Yolo**

I. Introduction

Agriculture is crucial to the development and stability of human society; however, insect pests cause enormous damage to global agricultural production every year, leading to reduced crop yields and severe economic losses. To effectively monitor pests during the critical crop growing season and to develop and implement efficient pest management strategies, advanced automation tools are urgently needed. Computer vision technology offers a powerful solution, with the potential to quickly and reliably automatically detect and identify insects by analyzing images and video recordings.

The goal of this project is to develop and compare several different computer vision methods for detecting and classifying insects in images of natural agricultural environments. All our experiments are based on the AgroPest-12 public dataset, which contains 12 insect categories and provides presplit image sets for 11502 images in training, 1095 images in validation, and 546 images in testing.

To systematically evaluate the effectiveness of different methods, our research is divided into two main sections. First, we conducted preliminary experiments using classic machine learning methods. We performed feature extraction based on SIFT (Scale Invariant Feature Transform) features and the Bag of Visual Words (BoVW) model. After feature extraction, we trained and tested five different classifiers (SVM, KNN, Random Forest, XGBoost, and MLP).

In our experiments with classic methods, MLP (Multilayer Perceptron) performed best, prompting us to further explore more powerful end-to-end deep learning neural network architectures. We then implemented a two-stage detector, Faster R-CNN, based on ResNet-50-FPN. Experiments showed that although its training speed was very slow, its detection accuracy was significantly improved compared to MLP. To further improve training efficiency, we evaluated a one-stage detector, RetinaNet, which is also based on the ResNet-50-FPN backbone and supplemented with data augmentation. We found that this method not only trained faster but also achieved significantly higher accuracy. Finally, to seek further performance optimization, we tried the efficiency-renowned one-stage detector, YOLO.

II. Literature Review New

This chapter reviews the evolution of object detection technology from classical feature engineering to deep learning, and focuses on the theoretical basis and technical context of model selection in this research.

A. Classical Methods

The Era of Feature Engineering Before deep learning became dominant, object detection mainly relied on a combination of hand-designed feature descriptors and machine learning classifiers. Early studies widely used descriptors such as Scale Invariant Feature Transform (SIFT) [5] and Histogram of Oriented Gradients (HOG) to capture local texture and shape information. In order to aggregate these local features into a fixed-length representation for image classification, the Bag-of-Visual-Words (BoVW) model [6] was widely adopted.

These features are usually fed into classifiers such as Support Vector Machine (SVM) [12] or Random Forests [14]. Although Multilayer Perceptron (MLP) has shown excellent nonlinear fitting ability in classical classifiers, these methods generally suffer from the "semantic gap" problem. They rely heavily on low-level visual cues and lack the ability to learn high-level semantic representations, thus limiting their robustness under complex backgrounds, occlusions and lighting changes [2]. This limitation has prompted research to turn to deep convolutional neural networks (CNNs) that can perform end-to-end feature learning.

B. Evolution of Two-Stage Detectors

In order to obtain feature representations that are superior to MLPs, we first focus on the development of two-stage detectors, which is an evolutionary path that pursues the ultimate detection accuracy.

R-CNN (Regions with CNN features): R-CNN [7], proposed by Girshick et al. in 2014, is the pioneering work of applying convolutional neural networks (CNNs) to object detection. It first uses the Selective Search algorithm to extract candidate regions, then extracts features through CNNs and uses SVMs for classification. Although R-CNN significantly outperforms the traditional HOG method, its main bottleneck is that it requires independent CNN forward computation for each candidate region, resulting in extremely redundant computation and a cumbersome training process (multi-stage pipeline).

Fast R-CNN: To solve the computational bottleneck of R-CNN, Girshick subsequently proposed Fast R-CNN [8]. This method introduces RoI Pooling (Region of Interest Pooling) layers, allowing the network to perform a convolution operation on the entire image only once and extract region features at the feature map level. This improvement achieves feature sharing, greatly improves the training and inference speed, and integrates the classifier and bounding box regressor into the same network. However, it still relies on external, computationally slow selective search algorithms to generate candidate boxes, which becomes the last bottleneck in system speed.

Faster R-CNN: Faster R-CNN [3] proposed by Ren et al. in 2015 finally completed this series of evolutions. This model introduces a Region Proposal Network (RPN) to replace selective search. RPN is a fully convolutional network that can share convolutional features with the detection network, thereby generating high-quality candidate regions at almost "zero cost". Faster R-CNN was the first to achieve end-to-end training and has long been regarded as the state-of-the-art baseline in the field of object detection.

- Reason for selection: Given the dominance of Faster R-CNN in two-stage detectors and its excellent feature extraction capabilities, this study first chose it as the starting point for deep learning methods to explore the upper limit of accuracy on the AgroPest-12 dataset.

C. Breakthrough of single-stage detectors: Why choose RetinaNet?(The Single-Stage Breakthrough)

Although Faster R-CNN has extremely high accuracy, its complex two-stage processing mechanism leads to slow inference speed, which is difficult to meet the needs of real-time agricultural monitoring. In order to seek a balance between efficiency and accuracy, we investigated single-stage detectors.

The dilemma of single-stage detectors: Although early single-stage detectors (such as YOLOv1 [11], SSD) are fast, their detection accuracy is often inferior to Faster R-CNN. Literature review shows that this accuracy gap is not due to insufficient network capacity, but rather to extreme class imbalance [10]. During the training of a single-stage detector, tens of thousands of candidate boxes (Anchors) may be generated in the image, most of which are easily classified background (negative samples). These large number of simple negative samples dominate the gradient update, causing the model to be unable to effectively learn sparse positive samples containing insects.

RetinaNet and Focal Loss: To address this core pain point, Lin et al. proposed RetinaNet [10]. This model did not design a more complex network structure, but retained a simple single-stage design and introduced two key components:

- Feature Pyramid Network (FPN): which enhances the ability to express features at multiple scales (especially small targets) [9]. Focal Loss: This is the core contribution of RetinaNet.
- Focal Loss modifies the standard cross-entropy loss function by adding a modulation factor $(1 - p_t)^\gamma$, which automatically reduces the weight of a large number of simple background samples, forcing the model to focus on training those samples that are difficult to classify (i.e., foreground targets).

Reason for selection: The literature results show that RetinaNet can achieve accuracy that surpasses Faster R-CNN at a single-stage speed after introducing Focal Loss. Therefore, in order to solve the speed bottleneck we encountered in the Faster R-CNN experiment and to verify the effectiveness of Focal Loss in the imbalanced scenario of agricultural pest data, we selected RetinaNet as the next step in model optimization.

D. YOLO family: From Foundations to Edge Performance In the field of deep learning object detection

The YOLO (You Only Look Once) family has become the first choice for real-time application scenarios with its extreme inference speed and continuously optimized accuracy. Its development process is a reflection of constantly seeking a better balance between speed and accuracy.

YOLOv1-v3: Foundation of infrastructure and real-time performance: Redmon et al. first proposed YOLOv1 in 2016 [11], which pioneered the view of object detection as a regression problem, directly predicting bounding boxes and categories from the whole image. This concept completely abandoned the region proposal mechanism and laid the foundation for single-stage real-time detection. The subsequent

YOLOv2 (YOLO9000)[15] introduced techniques such as Anchor Boxes and Batch Normalization to improve detection accuracy. YOLOv3[16] adopted the Darknet-53 backbone and multi-scale prediction to further improve the detection capability of small targets. The YOLO version at this stage established its leading position in real-time performance, but there is still a certain gap in accuracy compared to Faster R-CNN and RetinaNet.

YOLOv4-v5: Performance optimization and engineering practice: In order to make up for the accuracy gap and further improve efficiency, the YOLO series carried out a lot of engineering optimization and architectural innovation in v4[17] and v5[18] stages: CSPDarknet backbone: The CSPNet (Cross Stage Partial Network) idea was introduced, and the Darknet backbone was optimized, which reduced the amount of computation while maintaining or even improving the feature extraction capability. PANet neck structure: It integrates the Path Aggregation Network, which enhances multi-scale feature fusion and significantly improves the robustness of detection for targets of different sizes. Focus module: YOLOv5 introduced the Focus module (early version), which efficiently downsamples images and aggregates spatial information without increasing computational complexity.

YOLOv8: Towards Anchor-free and more efficient deployment: YOLOv8 [19] represents the evolution of the YOLO series towards a more modern architecture. Its core improvements include: Anchor-free design: It abandons the traditional Anchor Boxes and adopts the center point prediction method, which simplifies the training process, reduces hyperparameters, and further improves the flexibility and deployment efficiency of the model.

YOLOv11: The latest YOLOv11 [20] continues to innovate on the basis of its predecessor, introducing more efficient feature extraction units and continuously optimizing the performance of the model on various hardware platforms. This reflects the efforts of the YOLO series in continuously exploring lighter, more efficient and better performing target detection solutions.

III. Methods

This section covering classic machine learning pipelines and advanced deep learning architectures. All experiments were conducted on the AgroPest-12 dataset and strictly to the evaluation indicator.

A. Dataset and Evaluation Metrics

This research used the AgroPest-12 public dataset for model development and evaluation. This dataset contains 11,502 training images, 1,095 validation images, and 546 test images, covering 12 insect targets in natural agricultural environments. Each image includes accurate bounding boxes and category labels. To make sure the objectivity of the experimental results, we strictly follow to the pre-defined dataset splitting criteria, using only the training set for model parameter learning, the validation set for hyperparameter tuning, and the test set only used for final performance evaluation. We adopted the following standard evaluation metrics:

- Detection Performance: Mean Average Precision (mAP) which used as the core metric to measure the overall performance of object detection.
- Classification Performance: Precision, Recall, F1-Score, and Accuracy which calculated to evaluate classification ability.
- Efficiency: The training time and inference time per image for each model.

B. Data Imbalance Handling

We observed a significant imbalance in the sample distribution among different insect categories. And implemented a resampling strategy during the training phase. Specifically, we oversampling the minority class samples, increasing their sampling probability in each training batch to ensure the model learns feature representations for all classes in a balanced way.

C. Machine Learning Pipeline

To establish a performance baseline, we designed and implemented a classic machine learning pipeline consisting of three stages: feature extraction, bag-of-visual-words (BoVW) construction, and classifier discrimination.

- Feature Extraction and Bag-of-Virtual-Words (BoVW):
  - SIFT Feature Extraction: The Scale Invariant Feature Transform (SIFT) algorithm is used to detect keypoints and extract local descriptors from grayscale images. SIFT is rotation-invariant and scale-invariant, making it suitable for capturing textural details on insect surfaces.
  - Visual Vocabulary Construction: The K-Means clustering algorithm is used to cluster all SIFT descriptors in the training set, generating K cluster centers that constitute the "Visual Vocabulary".
  - Vector Quantization: Based on the Bag-of-Words (BoVW) model, the SIFT descriptor of each image is mapped to the nearest visual word, and word frequencies are counted to generate a fixed-length histogram feature vector.
- Classifier: The extracted BoVW feature vectors are input into five different machine learning classifiers for training and evaluation, including: Support Vector Machine (SVM), K-Nearest Neighbors (KNN), Random Forest, Extreme Gradient Boosting (XGBoost), and Multilayer Perceptron (MLP).

D. Data Augmentation Strategy

To improve the model's generalization ability, reduce overfitting, and better adapt the model to the complex and varied scenes in agricultural pest images, we implemented a systematic set of basic image augmentation strategies in the training data. Specific operations include:

- Random Horizontal Flip: Simulates the different orientations of insects in nature.

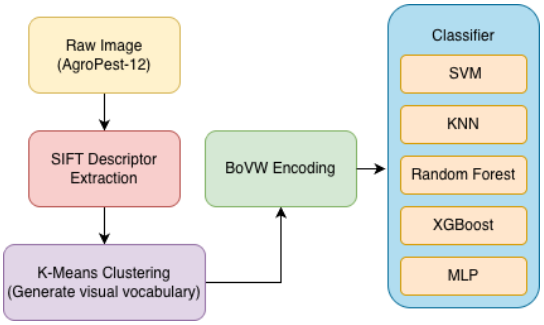


Fig. 1. A classic machine learning pipeline based on SIFT features and a bag-of-words (BoVW) model. This process separates feature engineering from the classifier array to establish a performance baseline.

- Random Brightness/Contrast Jitter: Simulates lighting conditions in the field at different times and under different weather conditions.
- Random Resize and Crop: Enhances the model's robustness.
- Image Normalization: Exposing the model to samples of different angles, lighting, and sizes during training that significantly improving its adaptability to real-world agricultural tasks.

E. Deep Learning Pipeline

To solve the limitations of handcrafted features and achieve end-to-end object detection, we evaluates three deep learning models.

F. Two-Stage Detector(Faster R-CNN)

Architecture: Region Proposal Network (RPN) generates high-quality candidate regions, followed by refined classification and regression.

Backbone: ResNet-50-FPN(Feature Pyramid Network) as the backbone which provides high-level semantic features with low-level high-resolution features.

Fig. 2. The Faster R-CNN's architecture.

G. Single-Stage Detector(RetinaNet)

To improve detection we chose RetinaNet. This is a single-stage object detector based on Anchor Regression, whose core architecture consists of the following three parts, aiming to address the challenges faced by traditional single-stage models.

a) Focal Loss: RetinaNet uses the focal loss to address extreme class imbalance. The binary focal loss is defined as:

$$FL(p_t) = -\alpha(1-p_t)^{\gamma}\log(p_t)$$

where

$$p_t = \begin{cases} p, & \text{if } y = 1 \\ 1-p, & \text{if } y = 0. \end{cases}$$

Here, p is the predicted probability for the positive class, α is the balancing factor, and γ is the focusing parameter.

b) Anchor Encoding and Decoding: For an anchor box with center (x_a, y_a) and size (w_a, h_a), the encoded regression targets are:

$$t_x = \frac{x - x_a}{w_a}, \qquad t_y = \frac{y - y_a}{h_a},$$

$$t_w = \log\left(\frac{w}{w_a}\right), \quad t_h = \log\left(\frac{h}{h_a}\right).$$

Decoding back to predicted box coordinates:

$$x = t_x w_a + x_a, \qquad y = t_y h_a + y_a,$$

$$w = e^{t_w} w_a, \qquad h = e^{t_h} h_a.$$

c) Feature Pyramid Network (FPN): The top-down pathway combines high- and low-resolution features:

$$P_5 = \text{Conv}(C_5),$$

$$P_l = \text{Conv}(C_l) + \text{Upsample}(P_{l+1}), \quad l \in \{3, 4\}.$$

Feature Extraction Backbone (Backbone: ResNet-50): We use ResNet-50 as the backbone network and load pre-trained weights from the COCO dataset to accelerate model convergence. ResNet-50 has powerful feature extraction capabilities, responsible for learning rich feature representations from low to high levels, including texture, edges, and shape, from images.

Core Innovation: Focal Loss: RetinaNet's biggest innovation lies in introducing Focal Loss to solve the problem of severe imbalance between positive and negative samples. In ordinary detection tasks, the training process is easily dominated by a large number of easily classifiable simple background samples (negative samples), making it difficult for the model to converge. Focal Loss dynamically reduces the weight of simple background samples while increasing the weight of difficult-to-classify samples (i.e., the insects themselves), forcing the model to focus on training difficult samples, thus achieving accuracy comparable to two-stage detectors in a single-stage architecture.

Data Augmentation: To further improve the model's generalization ability, we introduced data augmentation techniques including random flipping, cropping, scaling, and color jitter during training [10].

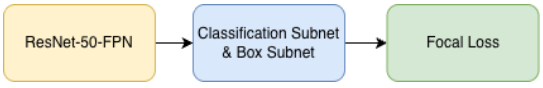


Fig. 3. The RetinaNet architecture uses the same ResNet-50-FPN backbone as Faster R-CNN and aims to improve the accuracy of single-stage detectors through Focal Loss.

H. YOLO(You Only Look Once)

- Core Idea: YOLO uses the CSPDarknet backbone and PANet neck structure, treating object detection as a single regression problem, directly predicting bounding boxes and categories from the entire image. This model aims to verify the possibility of achieving real-time inference while ensuring detection accuracy.

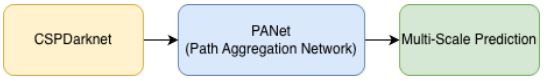


Fig. 4. The YOLO architecture, a single-stage detector based on regression principles, is renowned for its exceptional inference efficiency.

IV. Experimental Results

A. Performance Evaluation of Machine Learning Models

First, we examine the performance of the classical SIFT-BoVW pipeline from several perspectives, such as overall accuracy, class-level discriminative strength , and robustness under different perturbations.

a) Overall Accuracy Comparison: To build a baseline, we fed the SIFT-BoVW feature vectors into five different classifiers. The results below show that the MLP achieves the highest accuracy at 0.401 and also records the highest macro precision at 0.381 and the highest macro recall at 0.398. This is mainly because the SIFT-BoVW pipeline produces a high-dimensional and complex feature space, and the MLP—through its layered nonlinear structure—can capture the underlying decision boundaries more effectively. In contrast, models like KNN and Random Forest do not handle high-dimensional data as well.

b) ROC Curve and Discriminative Ability Analysis: To assess how confidently the model separates different insect classes, we examined the multi-class ROC curves and their corresponding AUC values. The MLP shows consistently strong discriminative performance, with AUC scores above 0.8 for classes such as bees, weevils, and ants. This indicates that the model can reliably distinguish true positives from false positives for these categories. Even for more challenging classes like beetles and slugs, the ROC curves remain smooth and noticeably above the chance line. These findings align with the accuracy results and further demonstrate the MLP's solid generalization ability across the dataset.

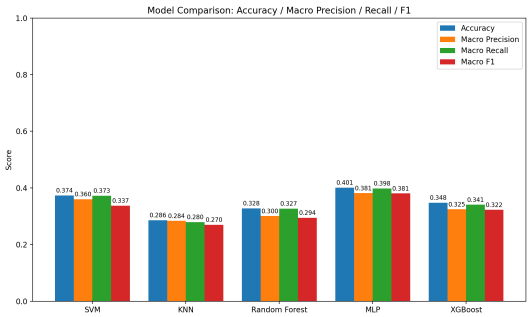


Fig. 5. ML Metrics Comparison.

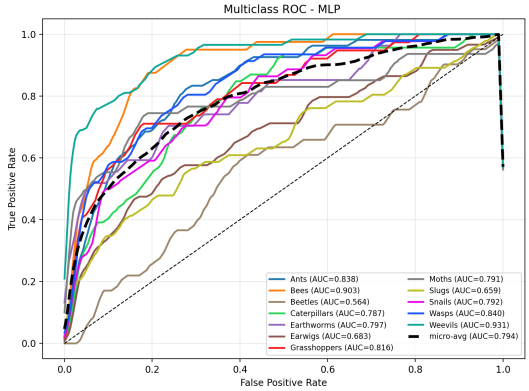
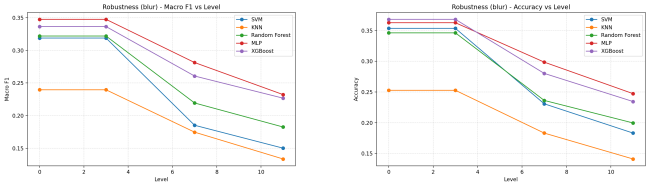


Fig. 6. ROC Curve(MLP).

c) Environmental Robustness Evaluation: To verify the model's reliability in non-ideal environments, we tested the model's performance under blurring, darkening, noise, and occlusion. The results are as below. MLP once again leads in overall stability:

- Blur and Occlusion: In Gaussian blur and partial occlusion scenarios, MLP's accuracy drops the most moderately. This demonstrates its network structure's robustness against local feature loss.
- Illumination Variation: In dimmed images, MLP performs relatively stably, even effectively utilizing residual shape and high-level features for prediction, without catastrophic failure.
- Noise Sensitivity: Although MLP's performance degrades rapidly under strong noise interference (indicating sensitivity to pixel-level perturbations), its extremely high baseline accuracy in noise-free environments ensures its final absolute performance remains superior. Overall, MLP not only achieves the highest accuracy on the standard test set but also demonstrates its practicality and stability as the best baseline model in complex and varied interference environments.



(a) Robust blur f1 score (b) Robust blur accuracy

Fig. 7. Model robustness under blur perturbation. Subfigure (a) shows the F1 score, and subfigure (b) shows the accuracy.

B. Evaluation of Deep Learning Models

To overcome the limitations of handcrafted features (MLPs achieve a maximum accuracy of only about 40%), we evaluated end-to-end deep learning models.

a) Faster R-CNN (Precision Baseline): As the accuracy baseline for the deep learning part, we first evaluated the two-stage detector Faster R-CNN with ResNet-50-FPN as the backbone, fine-tuning it based on the officially pre-trained weights. The FPN structure can fuse high-level semantics with low-level high-resolution features, thus balancing the detection capabilities of large-scale and small-to-medium-scale insect targets. During training, the SGD optimizer was used (initial learning rate 0.005, momentum 0.9, weight decay 0.0005), with a maximum of 30 training epochs, and the AP metric on the validation set was used as the early stopping criterion (patience value of 5 epochs). Meanwhile, to alleviate the severe inter-class long-tail distribution in AgroPest-12, we introduced RepeatFactorSampler during the PyTorch data loading stage to resample sample images corresponding to rare classes, thereby increasing the probability of minority classes appearing in each batch.

In the COCO-style validation set evaluation, Faster R-CNN achieved an overall average precision (AP) of 0.375, with AP50 = 0.726 at an IoU threshold of 0.5 and AP75 = 0.361 at a strict IoU threshold of 0.75. Breaking down by target size, the APs for small targets were approximately 0.000, the APm for medium targets was only 0.042, while the AP1 for large targets reached 0.391. This indicates that the model primarily relies on FPN to achieve relatively stable detection performance on large or well-defined insects, while remaining quite sensitive to extremely small-scale targets.

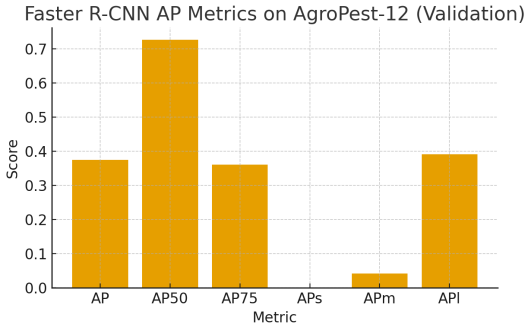


Fig. 8. Provide a bar chart summarizing the AP (primarily AP50) for each category.

It shows that categories with a larger sample size and larger target size have significantly higher APs, while categories with very few samples and small target sizes have APs close to 0. This phenomenon is highly consistent with data imbalance and the difficulty of detecting small targets.

As shown in Figure 9, the left image shows the manually labeled results (green bounding boxes) provided in the dataset, and the right image shows the prediction results of Faster R-CNN (red bounding boxes). The model assigns the category Weevils with a confidence score of approximately 0.96. It can be seen that on a grayscale background with complex textures, the predicted bounding box and the labeled bounding box highly overlap, indicating that the model can accurately capture the insect outline and provide a reliable

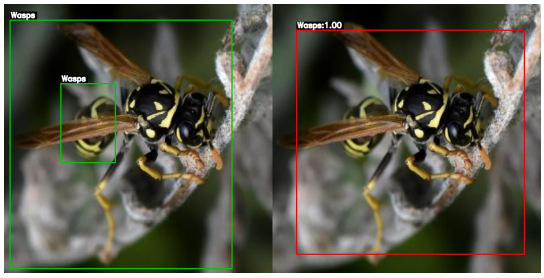


Fig. 9. To more intuitively understand the detection behavior of Faster R-CNN, we selected a representative Wasps image for visualization.

category judgment in scenes with single targets and large target sizes. Meanwhile, considering the lower AP values for other categories in Figure 8, it can be inferred that the main bottleneck of Faster R-CNN is not in the localization of single, sharp objects, but rather in insufficient recall for multiple objects, small objects, and samples with very few categories. This provides a starting point for subsequent improvements to single-stage architectures such as RetinaNet and YOLO.

b) RetinaNet (Single-Stage Detector): To address the dataset class imbalance problem in insect pest detection and try to improve inference efficiency, at that stage we trained a RetinaNet model with Focal Loss also using a ResNet-50-FPN backbone as the structure.

As the experimental evidence shown in Figure 10 and Figure 11, after 30 training epochs, the model demonstrated stable convergence and achieved competitive accuracy while maintaining relatively fast inference speed.

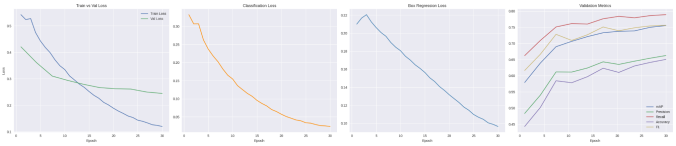


Fig. 10. RetinaNet Training and Validation Curves.

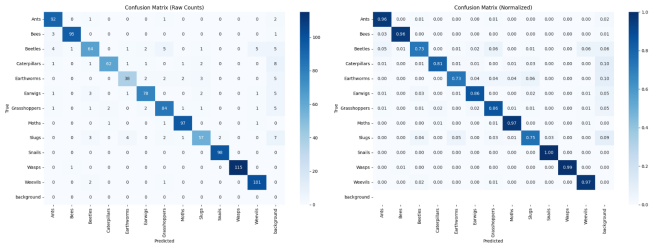


Fig. 11. RetinaNet Confusion Matrix (Raw & Normalized).

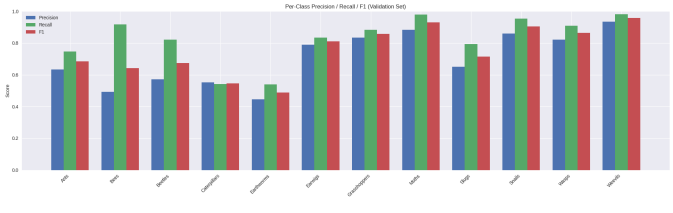


Fig. 12. RetinaNet Per-Class Validation Metrics.

The experimental results demonstrate that the method RetinaNet have achieved strong overall detection capability, with a validation mAP of 0.75 and an F1-score of 0.92. Benefiting from the Focal Loss design, the model effectively handled class imbalance and improved precision stability across insect categories well.

Although the precision (0.67) is moderate due to challenging small-object cases, the model also have the recall reached around 0.79, indicating the method successfully captured most ground-truth insect instances.At the same time, the high F1-score indicates a balanced and reliable detection performance across 12 pest classes from the dataset. The model training curves shows that smooth decline in total, about classification and box regression losses, mAP steadily increasing until 30 epochs, indicating convergence, and no signs of overfitting, as validation performance stabilizes along with losses, indicated that as a reasonable method.

Ablation on Data Augmentation: In the augmented setting, the training pipeline included random horizontal/vertical flips, scale jittering, color jitter (brightness and contrast), and random cropping.Augmentation notably improved robustness to changes in lighting,orientation, and also object size—common issues in field surveillance.

	mAP@0.5	F1-score
Model Without Augmentation	0.60	0.80
Model With Augmentation	0.75	0.92

TABLE I
PERFORMANCE COMPARISON OF AUGMENTATION IN RETINANET.

All experiments that way used identical hyperparameters and preprocessing settings, and we introducing geometric and photometric augmentation led to a clear mAP improvement, showing that data diversity significantly enhances from RetinaNet Model's generalization.

However, RetinaNet still presents structural limitations: the anchor-based design creates heavy class imbalance and unstable early training, and fixed anchor scales make small or elongated insects harder to detect relatively. Also the architecture is less flexible for integrating modern detection heads or advanced augmentations among all the methods.

c) Yolo: To evaluate the performance differences among YOLO11 models of different scales (n/s/m) on the insect detection task, this section conducts a horizontal comparison of the key evaluation metrics, followed by both quantitative and qualitative analyses.

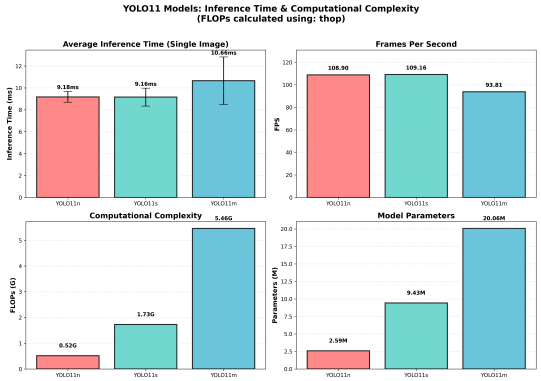


Fig. 13. Inference Efficiency Comparison.

Figure presents a comprehensive comparison of inference performance and computational complexity across three YOLO11 variants. YOLO11s achieved the highest FPS (109.16) with minimal difference from YOLO11n (108.90), despite 3.3× higher FLOPs. YOLO11n demonstrated superior computational efficiency (209.4 FPS/GFLOPs), making it optimal for edge deployment. All models exceeded real-time requirements (>90 FPS).An unexpected observation from our experiments is that YOLO11s achieved the highest inference speed of 109.16 FPS, marginally surpassing YOLO11n (108.90 FPS), despite requiring 3.3× more FLOPs (1.73 vs 0.52 GFLOPs). This counterintuitive phenomenon suggests that the additional computational operations in YOLO11s are better aligned with the target hardware architecture's optimization characteristics. This reveals YOLO's time efficiency

Model	Precision	Recall	F1-score	mAP50	mAP50-95	Accuracy
YOLO11n	0.8317	0.6805	0.7485	0.7401	0.4441	0.6335
YOLO11s	0.8574	0.7069	0.7749	0.7699	0.4713	0.6628
YOLO11m	0.8755	0.7066	0.7820	0.7802	0.4822	0.6387

TABLE II
PERFORMANCE COMPARISON OF YOLO11 MODEL VARIANTS.

From the table, the following observations can be made: Baseline detection capability (mAP50) increases steadily as the model size grows (YOLO11n → YOLO11s → YOLO11m: 0.7401, 0.7699, 0.7802). This indicates that a larger number of parameters helps the model capture more complex insect characteristics, thereby improving baseline detection accuracy. Detection stability (mAP50–95) also improves with model scale (0.4441 → 0.4713 → 0.4822), suggesting that larger models are more robust in strict bounding-box matching scenarios.

In terms of precision–recall balance (F1-score), YOLO11m achieves the highest value (0.7820), demonstrating the best trade-off between accurate insect identification and comprehensive recall of true targets. In contrast, YOLO11n has the lowest F1-score (0.7485), consistent with the previously observed issue where Earwigs are easily confused with Ants and Beetles, leading to a higher false-positive rate when multiple insects appear. This confirms that the smaller model struggles to distinguish between visually similar classes.

Our experiments revealed that YOLO11m exhibits overfitting behavior on small datasets, where the model learns overly specific features that fail to generalize to unseen data. To address this limitation, we explored data augmentation strategies.

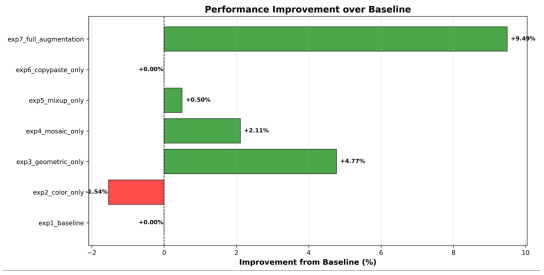


Fig. 14. Performance Improvement over Baseline.

Color augmentation degraded performance: mAP50 dropped from 0.6692 to 0.6585, and mAP50–95 from 0.4056 to 0.3994, despite precision gains (0.7418→0.7737). This suggests color information is critical for insect detection, making color transformations unsuitable.

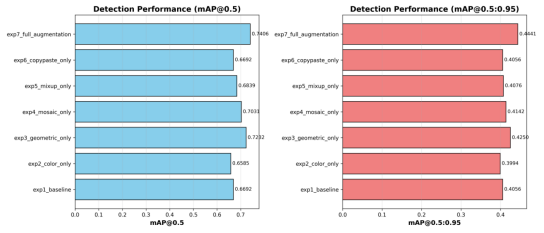


Fig. 15. Detection Performance.

Geometric augmentation yields the most significant gains: mAP50 rises from 0.6692 to 0.7232 (+0.014) , mAP50–95 improves from 0.4056 to 0.4250 (+0.0478), and all precision/recall metrics increase. By expanding pose and spatial diversity, geometric transformations effectively strengthen the model's representation of insect structures.

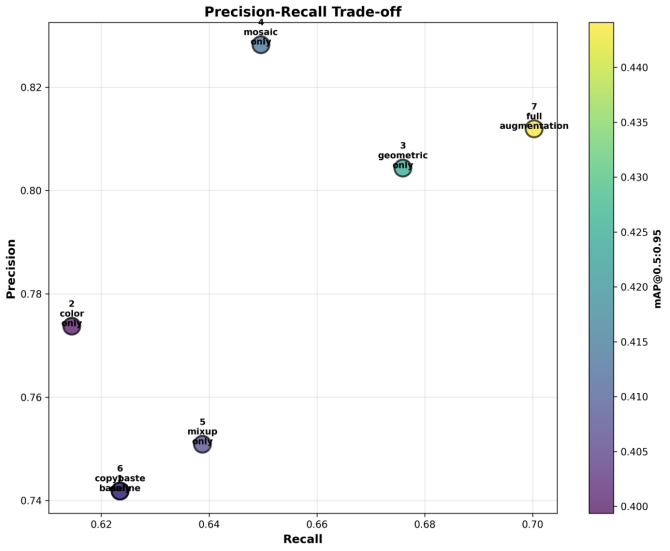


Fig. 16. Precision-Recall Trade-off.

Different augmentation strategies exhibit complementary effects across spatial robustness, scene complexity, and sample diversity. Their combination significantly outperforms individual techniques, validating the benefits of multi-augmentation synergy.

V. Discussion

This section about analysis of the experimental results. First, it explores the advantages and limitations of Multilayer Perceptrons in classic pipelines, then introduces the fundamental differences between deep learning and traditional methods. Finally, it details the architectural evolution and performance within deep learning models, and discusses the practical challenges and solutions encountered in the experiments.

A. Performance of MLPs and the Limits of Neural Networks

Classic machine learning limited by the limitations of handcrafted feature like SIFT-BoVW features. MLPs consistently outperform other classifiers such as SVMs, Random Forests, and XGBoost. This result shows that even with fixed handcrafted features as input, neural network structures still get low accuracy. MLPs use nonlinear activation functions of their hidden layers which can better fit the complex and nonlinear decision boundaries in the feature space better than traditional linear classifiers.

However, despite MLP's leading position among classic methods, its overall performance (Accuracy/F1-Score) still has a significant ceiling. This is because MLP is essentially just a classifier, relying on fixed SIFT feature extraction at the front end. While SIFT is sensitive to local texture, it lacks understanding of semantic context, and the quantization process of BoVW loses spatial structural information. Therefore, the limitations of MLP actually reflect the bottleneck of "handcrafted feature engineering."

B. Transition from Handcrafted Features to End-to-End Learning

Experimental data shows that the detection performance (mAP) of all deep learning models (Faster R-CNN, RetinaNet, YOLO) significantly surpasses the combination of "SIFT-BoVW + MLP." This leap forward is attributed to a fundamental change in feature representation. While MLPs are neural networks, they can only process shallow features that are "fed" to them. Deep Convolutional Neural Networks (CNNs), on the other hand, achieve end-to-end feature learning from raw pixels to high-level semantic features through their deep hierarchical structure. Deep models can not only capture texture but also understand shape, the relationship between parts and the whole, and contextual information, thus exhibiting far greater robustness than handcrafted features when faced with changes in lighting, occlusion, and complex backgrounds.

C. Evolution and Trade-offs in Deep Architectures

Having established the dominance of deep learning, we further analyze the evolutionary logic from two-stage to single-stage, and then to efficient single-stage architectures:

Accuracy Benchmark and Efficiency Bottleneck (Faster R-CNN): As a representative of two-stage detectors, Faster R-CNN utilizes RPN to generate high-quality candidate boxes, setting a high accuracy benchmark. However, its complex two-stage sequential processing mechanism leads to high computational costs and slow inference speeds, making it difficult to meet the needs of real-time applications.

Single-Stage Overtaking and Limitations (RetinaNet): In experiments, RetinaNet achieved higher mAP when using the same ResNet-50-FPN backbone as Faster R-CNN. This counterintuitive improvement is attributed to the introduction of Focal Loss, which elegantly addresses the extreme foreground-background class imbalance problem faced by single-stage detectors at the loss function level. However, during experiment we also found limitations of the RetinaNet architecture:

- Inefficiency of Anchor-based Structure: The anchor box generates a massive number of invalid negative samples. Resulted in slow training convergence.
- Insufficient Inference Speed: Although faster than Faster R-CNN, its still not good.
- Robustness Bottleneck: The model still suffers from missed detections of extremely small targets and influence high sensitivity to lighting conditions and shooting angles in the dataset.
- Limited Scalability: The model structure is relatively rigid which making it difficult to flexibly extend to new modules or integrate complex data augmentation strategies.

Addressing the limitations of RetinaNet, we final chose to upgrade to the YOLO architecture. YOLO demonstrated the best overall performance in this research, with advantages including:

- Efficient Detection Mechanism: YOLO eliminates cumbersome anchor box calculations, significantly saving training stability and convergence speed.
- Advanced Data Augmentation: YOLO supports advanced augmentation strategies which significantly improves the model's adaptability to small objects and robustness to complex backgrounds.

D. Limitations Analysis and Future Work

We found some key limitations affecting model performance during experiments: Redundant Detection, Label Ambiguity, and the Dilemma of Merging Strategies.

- Multiple Detection Problem: We observed that the same target insect was frequently predicted with multiple overlapping bounding boxes during the model training stage. This led to wrong target counts which cause fail to completely eliminate redundancy in certain dense or ambiguous scenarios.
- Box Merging Strategy and Reasons for Its Failure to Implement: To completely resolve the duplicate counting problem, we initially planned to implement a Box Merging Strategy, which involves weighted merging of multiple overlapping predicted boxes targeting the same object during post-processing to ensure that each object corresponds to only one detection result.
- Dataset Constraints: However, this strategy was ultimately not deployed in this phase of the experiments. The main reason is that we discovered systematic annotation ambiguity in the dataset itself: in the ground truth annotations of the training set, there is a common phenomenon where "a single object is labeled by multiple independent or overlapping bounding boxes simultaneously." In this data context, forcibly merging detection boxes only at the inference stage would lead to a mismatch between the "correct" single boxes generated by the model and the "redundant" ground truth boxes in the dataset, potentially resulting in misclassification as incorrect when calculating mAP.

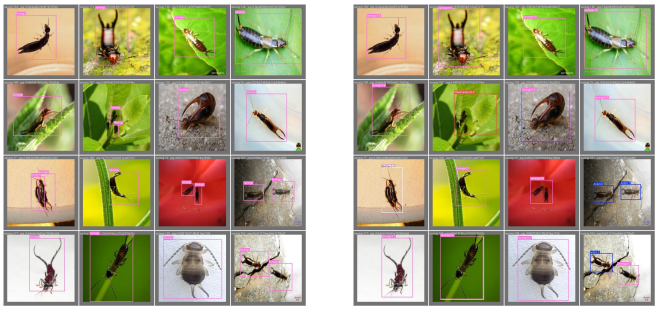
Background and Class Confusion: Against complex backgrounds such as foliage or soil textures similar in color to insects. The model can't dentifies non-insect areas as targets, leading to false alarms. What's more, we observed specific inter-class confusion phenomena:

- Earwig's Recognition: Earwigs are frequently confused with ants. Especially in complex situations where multiple earwigs appear in the image. The model misidentifying some earwigs as beetles, ants, or even grasshoppers. This may be related to the morphological similarity of these insects from certain perspectives or insufficient diversity of samples.

Insufficient annotation of organism life cycle: Such as caterpillars , they varies at different life stages like arva, adult and pupa. But they may be labeled as "caterpillar" in the dataset which makes it difficult for the model to learn the unique features of each stage, thus reducing the accuracy.

E. Future Work Outlook

We will focus on the following directions in the future:



(a) Original (b) Prediction

Fig. 17. Wrong detection in Yolo.

Data Cleaning and Relabeling: The primary task is to conduct a review and correction of existing datasets. Slove redundant bounding boxes for individual targets. More precise labeling of individuals at different times. Based on this, implementing a bounding box merging strategy will truly improve counting accuracy.

Fine-grained Feature Learning: For easily confused categories such as earwigs and ants, explore finer-grained feature extraction methods or introduce contrastive learning to enhance the model's ability to distinguish similar categories.

Small Target and Camouflage Robustness: Explore more advanced feature pyramid structures, contextual attention mechanisms, or high-resolution detection heads to enhance the ability to identify small and highly camouflaged targets.

Generative Data Augmentation: Utilize GANs to generate pest samples in rare or highly camouflaged scenarios to further address the long-tail distribution problem.

Model Interpretability: Introduce interpretable AI technologies (such as Grad-CAM) to visualize the model's decision-making process and increase agronomic experts' trust in automated systems.

VI. Conclusion

This research aims to address the problem of automatic insect detection and classification in natural agricultural environments. We have comprehensively compared various computer vision methods from classical machine learning to deep learning.

The experimental results from machine learning show that MLP (Multilayer Perceptron) which with nonlinear fitting capabilities is better than traditional linear classifiers when processing handcrafted features. But its overall performance is still limited by the expressive bottleneck of handcrafted features. This finding drives our towards to end-to-end feature learning.

We reveal the evolutionary logic of different architectural designs through comparative experiments in the deep learning phase:

- Faster R-CNN have a great accuracy but its two-stage architecture limits inference speed.
- RetinaNet use Focal Loss successfully overcame the class imbalance disadvantage of single-stage detectors and surpassing two-stage models in accuracy.
- YOLO ultimately achieved the optimal balance between speed and accuracy. Leveraging its anchor-free mechanism and efficient feature aggregation network, YOLO achieves real-time inference while maintaining the highest mAP, proving itself to be the most suitable solution for real-time agricultural monitoring tasks.

What's more, we find future improvements should not be limited to simply stacking model architectures, but should return to the data itself. Overcoming current performance bottlenecks through higher-quality data cleaning, re-annotation, and the implementation of bounding box merging strategies.

References

[1] R.Majumdar, "AgroPest-12: Image Dataset for Crop Pest Detection," Kaggle, Sep. 2025. [Online].

[2] S. Chakrabarty et al., "Application of artificial intelligence in insect pest identification - a review," Artificial Intelligence in Agriculture, vol. 16, no. 1, pp. 44-61, Mar. 2026.

[3] K. Wang et al., "AP162: A large-scale dataset for agricultural pest recognition," Computers and Electronics in Agriculture, vol. 237B, p. 110520, Oct. 2025.

[4] X. Wu et al., "IP102: A large-scale benchmark dataset for insect pest recognition," in IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), Jun. 2019.

[5] D. G. Lowe, "Distinctive image features from scale-invariant keypoints," International Journal of Computer Vision, vol. 60, no. 2, pp. 91–110, 2004.

[6] G. Csurka, C. R. Dance, L. Fan, J. Willamowski, and C. Bray, "Visual categorization with bags of keypoints," in Workshop on Statistical Learning in Computer Vision, ECCV, vol. 1, no. 1-22, pp. 1–2, 2004.

[7] R. Girshick, J. Donahue, T. Darrell, and J. Malik, "Rich feature hierarchies for accurate object detection and semantic segmentation," in IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2014.

[8] R. Girshick, "Fast R-CNN," in IEEE International Conference on Computer Vision (ICCV), 2015.

[9] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature Pyramid Networks for Object Detection," in IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2017.

[10] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal Loss for Dense Object Detection," in IEEE International Conference on Computer Vision (ICCV), 2017.

[11] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016.

[12] C. Cortes and V. Vapnik, "Support-vector networks," Machine Learning, vol. 20, pp. 273-297, 1995.

[13] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016.

[14] L. Breiman, "Random Forests," Machine Learning, vol. 45, pp. 5-32, 2001. (Random Forest

[15] J. Redmon and A. Farhadi, "YOLO9000: Better, Faster, Stronger," in IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2017.

[16] J. Redmon and A. Farhadi, "YOLOv3: An Incremental Improvement," arXiv preprint arXiv:1804.02767, 2018.

[17] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal Speed and Accuracy of Object Detection," arXiv preprint arXiv:2004.10934, 2020.

[18] G. Jocher et al., "YOLOv5 by Ultralytics," GitHub repository, 2020. [Online].

[19] G. Jocher et al., "YOLOv8 by Ultralytics," GitHub repository, 2023. [Online].

[20] G. Jocher et al., "Ultralytics YOLO11," GitHub repository, 2024. [Online].